

DHL

DAC 3.0 CHALLENGE

AI-Enhanced Incident Reporting & Resolution System

SECJ3483 Web Technology | Scenario 2 | Section 3

HANAN OSAMA HUSSEIN SALAH
A22EC4042

Academic Session 2025/2026 — Universiti Teknologi Malaysia (UTM)

1. Introduction and Scenario Description

1.1 Background

This assignment is conducted in collaboration with DHL Global Finance Services – Asia Pacific through the DAC 3.0 Challenge. The objective is to design and develop a real-world web application integrated with Robotic Process Automation (RPA) to solve a logistics operational problem.

1.2 Scenario Description — Scenario 2

DHL's Customer Support teams receive a high volume of incident reports every day from multiple inconsistent channels including email inboxes, Telegram/Teams chat messages, phone call notes, images/screenshots of damaged packages, and handwritten instructions from warehouse teams. Because the information is unstructured, incomplete, and inconsistent, it is difficult for DHL to identify issues, assign incidents, prioritize correctly, and track resolution progress. This leads to slow response times, repeated manual work, and inconsistent customer service quality.

1.3 Objectives

The objective is to design and develop an AI-powered Incident Reporting System that automates the full process of collecting, understanding, assigning, and tracking incident reports:

- A Laravel-based RESTful API backend with PostgreSQL database
- A React-based frontend user interface with DHL branding
- A UiPath RPA workflow for automated incident ingestion and reporting
- An AI Draft Builder powered by Claude AI for intelligent incident structuring

2. System Architecture

2.1 Overall Architecture

The system follows a three-tier architecture consisting of a React frontend, a Laravel RESTful API backend, and a PostgreSQL database. The UiPath RPA workflow interacts with the system through the REST API. An AI layer powered by Claude (Anthropic) provides intelligent incident analysis.

- User Interface Layer: React (Vite) running on localhost:8080
- API Layer: Laravel 13 (PHP 8.4) running on localhost:8000
- Database Layer: PostgreSQL 16 running on localhost:5432
- RPA Layer: UiPath Studio 2026 interacting via HTTP requests
- AI Layer: Claude API (Anthropic) for incident analysis and structuring

2.2 Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite + TailwindCSS	User interface, forms, dashboard
Backend	Laravel 13 (PHP 8.4)	RESTful API, business logic, auth
Database	PostgreSQL 16	Data persistence
Authentication	Laravel Sanctum	Token-based API authentication
AI	Claude API (Anthropic)	AI Draft Builder & Conflict Check
RPA	UiPath Studio 2026	Automated incident ingestion
Version Control	GitHub	Source code management

2.3 Database Schema

Table	Key Fields	Purpose
users	id, name, email, password, role	System users (admin, agent, reviewer)
departments	id, name, email	DHL departments for assignment
incidents	id, title, description, type, source, status, priority, department_id, created_by	Core incident records
incident_logs	id, incident_id, action, old_status, new_status, changed_by	Audit trail for status changes

3. Technical Explanation (API, Data, JSON)

3.1 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/register	Register a new user	No
POST	/api/login	Login and get Bearer token	No
GET	/api/incidents	List all incidents (filterable)	Yes
POST	/api/incidents	Create a new incident	Yes
GET	/api/incidents/{id}	Get incident details + logs	Yes
PUT	/api/incidents/{id}	Update incident	Yes
PATCH	/api/incidents/{id}/status	Update incident status	Yes
DELETE	/api/incidents/{id}	Delete incident	Yes
GET	/api/departments	List all departments	Yes
POST	/api/departments	Create department	Yes

3.2 JSON Data Structure

Sample Incident JSON Response:

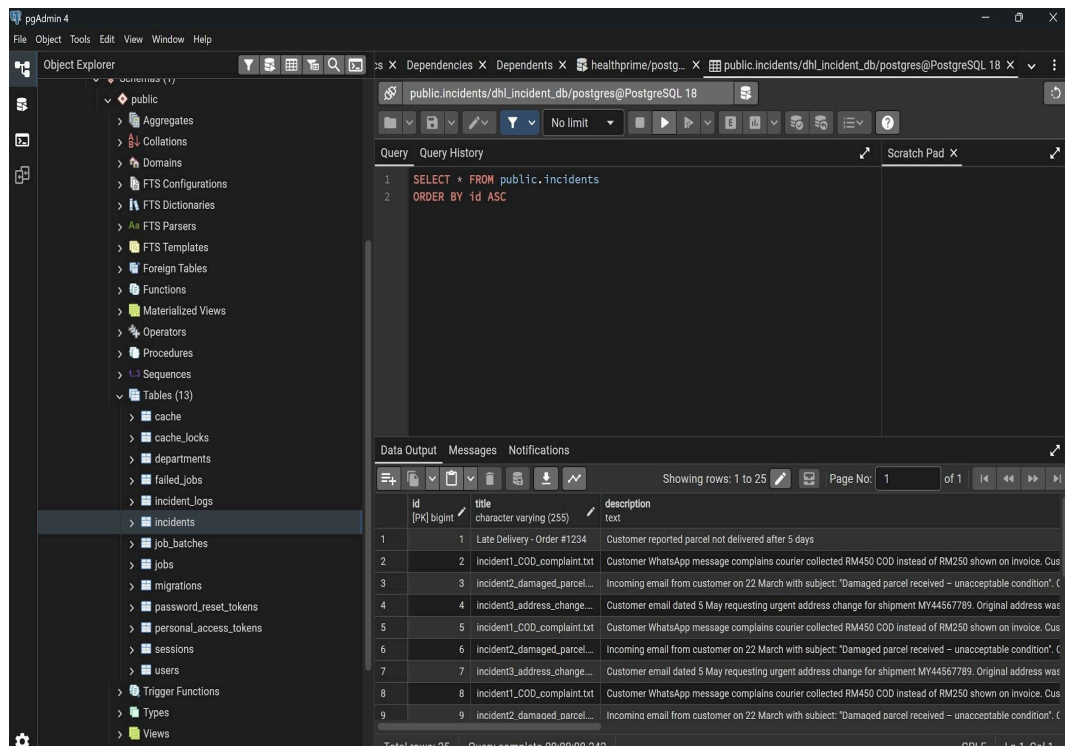
```
{ "id": 1, "title": "Late Delivery - Order #1234", "description": "Customer reported parcel not delivered", "type": "late_delivery", "source": "email", "status": "draft", "priority": "high", "created_at": "2026-05-15T04:47:45.000000Z" }
```

3.3 Authentication

The system uses Laravel Sanctum for token-based authentication. Upon login, the API returns a Bearer token included in all subsequent requests as an Authorization header: 'Authorization: Bearer {token}'. Tokens are stored per-user in the `personal_access_tokens` table.

3.4 Database — pgAdmin View

The PostgreSQL database `dhl_incident_db` contains 13 tables. The `incidents` table contains 25 records created through both manual entry and the RPA workflow automation.



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'incidents' table selected under the 'public' schema. The main pane shows a query window with the following SQL:

```
SELECT * FROM public.incidents
ORDER BY id ASC
```

The Data Output pane displays the results of the query, showing 25 rows. The table has three columns: 'id' (integer), 'title' (character varying), and 'description' (text). The data is as follows:

id	title	description
1	Late Delivery - Order #1234	Customer reported parcel not delivered after 5 days
2	Incident1_COD_complaint.txt	Customer WhatsApp message complains courier collected RM450 COD instead of RM250 shown on invoice. Cus
3	Incident2_damaged_parcel...	Incoming email from customer on 22 March with subject: "Damaged parcel received - unacceptable condition". C
4	Incident3_address_change...	Customer email dated 5 May requesting urgent address change for shipment MY44567789. Original address was
5	Incident1_COD_complaint.txt	Customer WhatsApp message complains courier collected RM450 COD instead of RM250 shown on invoice. Cus
6	Incident2_damaged_parcel...	Incoming email from customer on 22 March with subject: "Damaged parcel received - unacceptable condition". C
7	Incident3_address_change...	Customer email dated 5 May requesting urgent address change for shipment MY44567789. Original address was
8	Incident1_COD_complaint.txt	Customer WhatsApp message complains courier collected RM450 COD instead of RM250 shown on invoice. Cus
9	Incident2_damaged_parcel...	Incoming email from customer on 22 March with subject: "Damaged parcel received - unacceptable condition". C

The bottom status bar indicates 'Total rows: 25' and 'Query complete 00:00:00.242'.

Figure 11: pgAdmin 4 — Database tables and incidents data (25 rows)

4. RPA Workflow and Automation Explanation

4.1 RPA Overview

The UiPath RPA workflow (DHL_Incident_RPA) automates the process of ingesting incident files and creating structured incident records in the web application through the REST API.

4.2 Workflow Steps

Step	Activity	Description
1	Login & Auth	Sends POST to /api/login, extracts Bearer token from JSON response
2	Read Folder	Scans C:\DHL_Incidents folder for all .txt files
3	Duplicate Check	Reads processed_log.txt, skips files processed within last 14 days
4	Create Incident	POSTs to /api/incidents with file content, title, type, source, priority
5	Retry Mechanism	Retries failed requests up to 3 times with 5-second delay
6	Log Results	Writes timestamped entries to automation_log.txt
7	Summary Email	Sends SMTP email via Gmail with totals: created, skipped, failed

4.3 RPA Workflow — Part 1: Login & File Reading

The first section handles authentication with the Laravel API and retrieves all files from the DHL_Incidents folder. It uses a Try-Catch block for error handling, serializes the login body as JSON, sends the POST request, logs the raw response, deserializes the JSON, and stores the Bearer token.

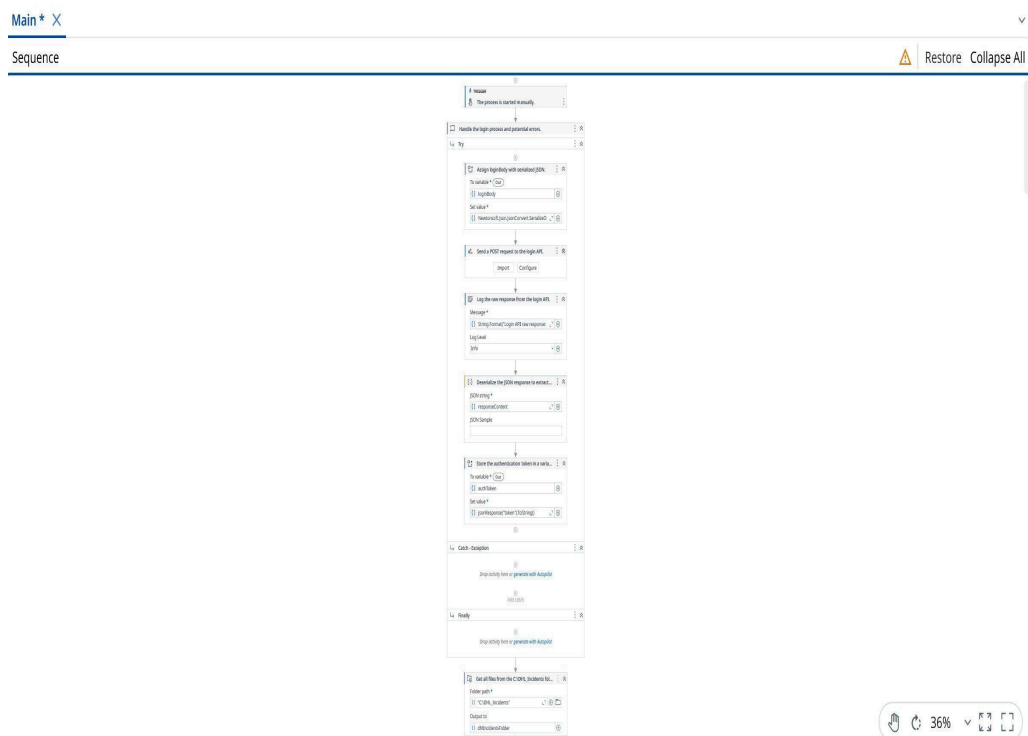


Figure 8: UiPath Workflow Part 1 — Login, token extraction and file reading

4.4 RPA Workflow — Part 2: File Processing & Duplicate Check

The second section iterates through each file using a For Each loop, checks the processed log for duplicates within 14 days, reads file content, initializes a retry counter, and processes new files by POSTing to the API.

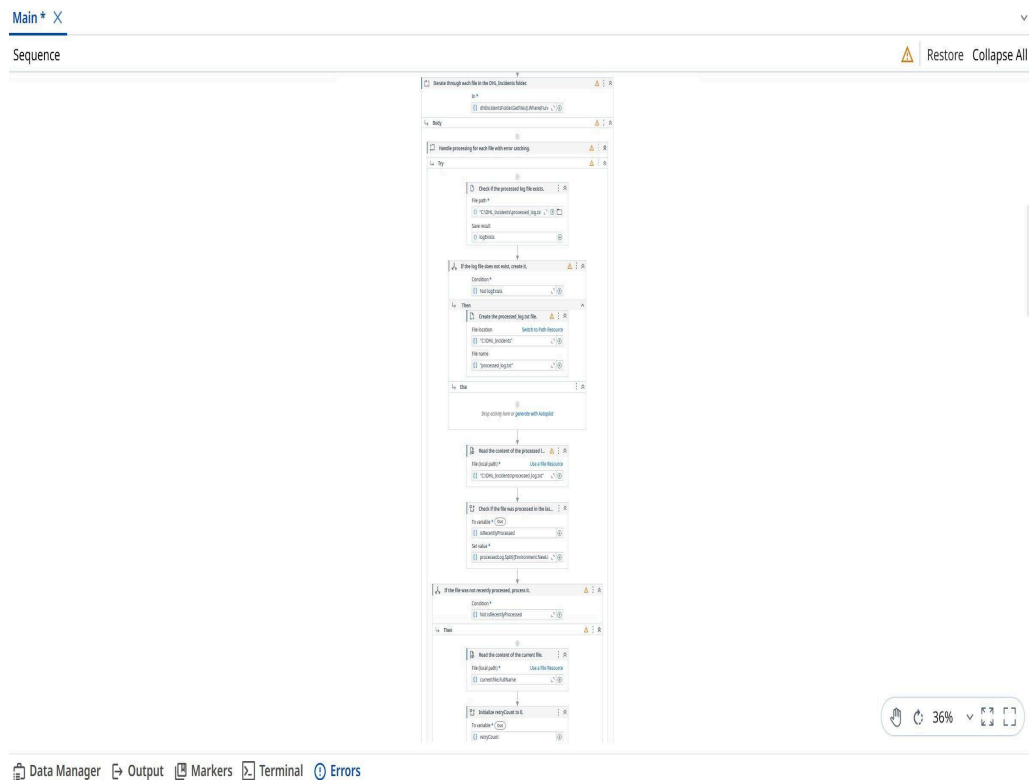


Figure 9: UiPath Workflow Part 2 — Per-file processing with duplicate check logic

4.5 RPA Workflow — Part 3: Summary Email

The final section constructs the summary email body with processing totals (created, skipped, failed), sends it via SMTP to the admin email with subject 'DHL Incident Automation Summary', and logs the result. A Catch block handles email failures gracefully.

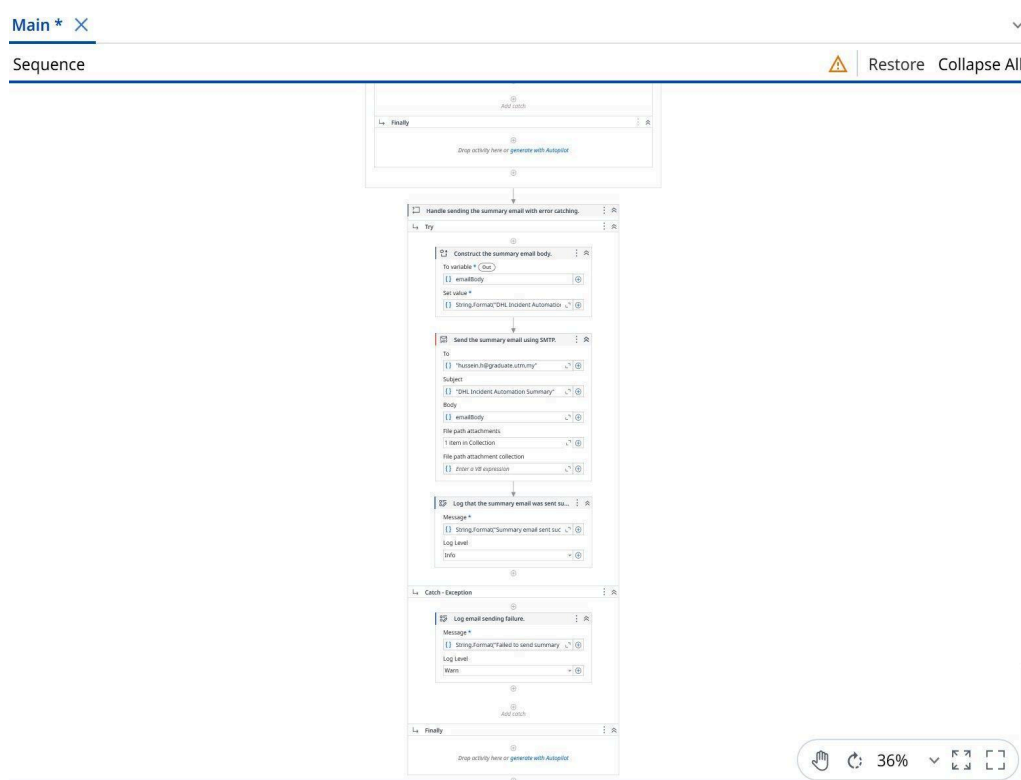


Figure 10: UiPath Workflow Part 3 — Summary email construction and SMTP sending

4.6 Error Handling

The workflow implements comprehensive error handling using Try-Catch blocks at every stage. On any failure, the workflow logs the error with timestamp, increments the failed counter, and continues processing remaining files. All results are summarized in a final email report.

5. User Interface Screenshots and Description

5.1 Login Page

The login page features DHL branding with the signature red and yellow color scheme. Users enter their email and password to authenticate via the Laravel Sanctum API. The Bearer token is stored in localStorage upon successful login.

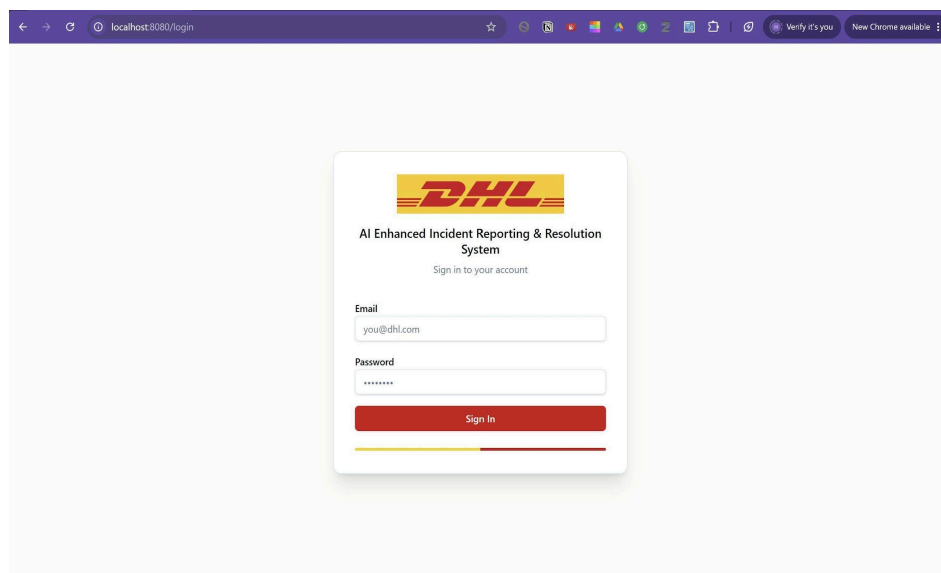


Figure 1: Login Page — DHL branded authentication

5.2 Landing / Home Page

The landing page presents the system with three key features: AI Triage for automatic incident classification, Unified Intake for multi-channel support (email, Teams, phone, WhatsApp), and Faster Resolution for tracking incidents across departments.

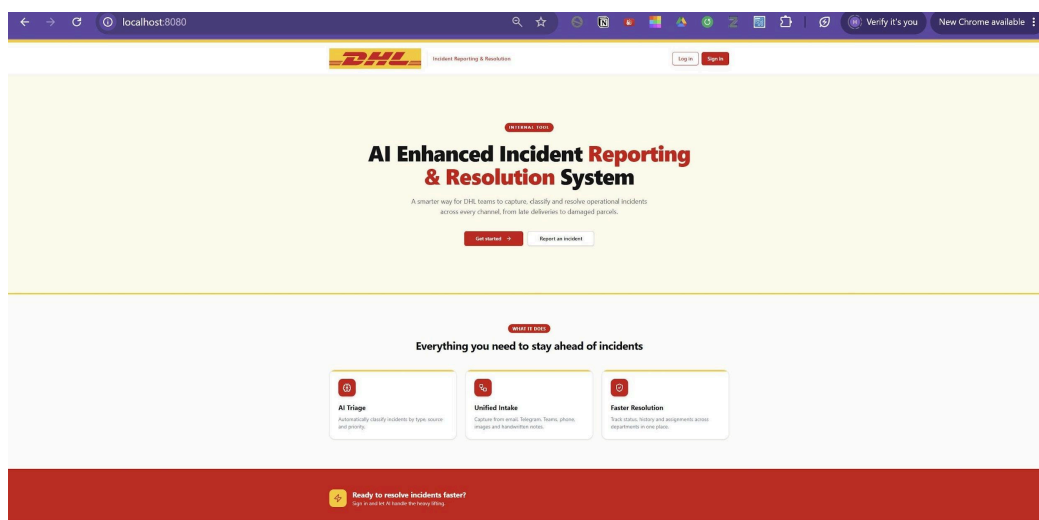


Figure 2: Landing Page — System overview with feature highlights

5.3 Incident Dashboard

The dashboard displays all incidents in a searchable, filterable list. Users can filter by status (Draft, Reviewed, Resolved), priority (Low, Medium, High, Critical), and incident type. Each entry shows its ID, type badge, title, description preview, and status/priority tags. Currently showing 24 of 24 incidents.

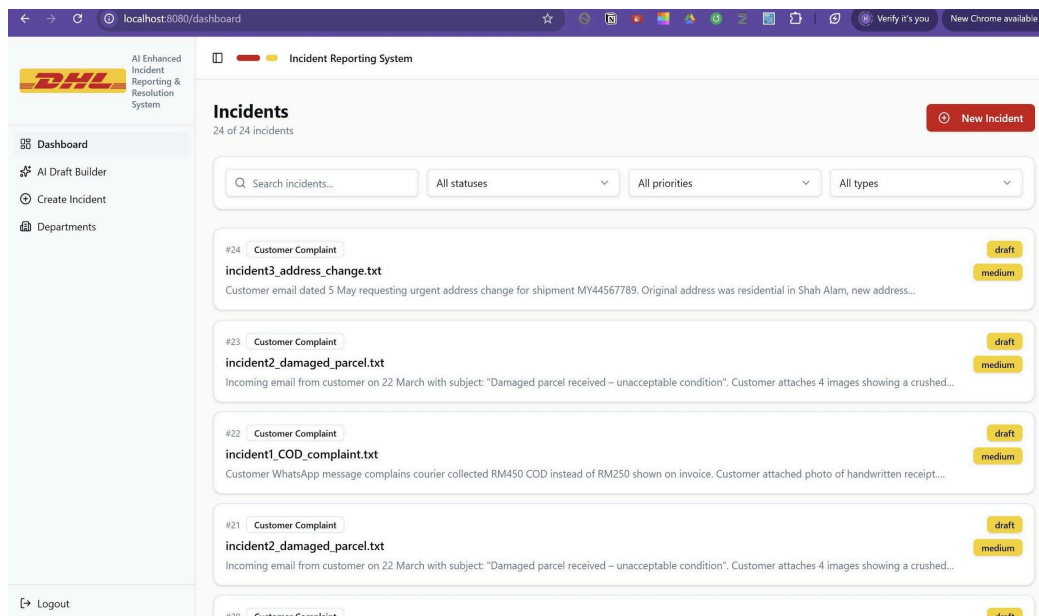


Figure 3: Dashboard — Incident list with search and filter functionality

5.4 Create Incident Page

The Create Incident form allows manual reporting with fields for Title, Description, Type (Late Delivery, Address Issue, Damaged Parcel, System Error, Customer Complaint), Source channel, Priority level, and an Attachment upload for images, documents, or audio files.

The screenshot shows a web browser at localhost:8080/incidents/create. The page title is 'Report New Incident' with the subtitle 'Provide as much detail as possible'. The form includes a 'Title' field with a placeholder 'Brief summary of the incident', a 'Description' field with a placeholder 'Provide detailed context, location, parcel IDs, customer info...', and three dropdown menus for 'Type' (placeholder 'Select type'), 'Source' (placeholder 'Select source'), and 'Priority' (placeholder 'Medium'). There is also an 'Attachment' section with a placeholder 'Click to upload an image, document, or audio file'. At the bottom right of the form are 'Cancel' and 'Submit Incident' buttons. The left sidebar contains the DHL logo, 'AI Enhanced Incident Reporting & Resolution System', and navigation links: 'Dashboard', 'AI Draft Builder', 'Create Incident' (active), and 'Departments'. A 'Logout' link is at the bottom of the sidebar.

Figure 4: Create Incident Page — Manual incident reporting form

5.5 AI Draft Builder — Input

The AI Draft Builder accepts raw unstructured text from any source (email, WhatsApp, phone notes, chat logs). Users paste the text and click 'Analyze with AI' to trigger the Claude AI analysis pipeline.

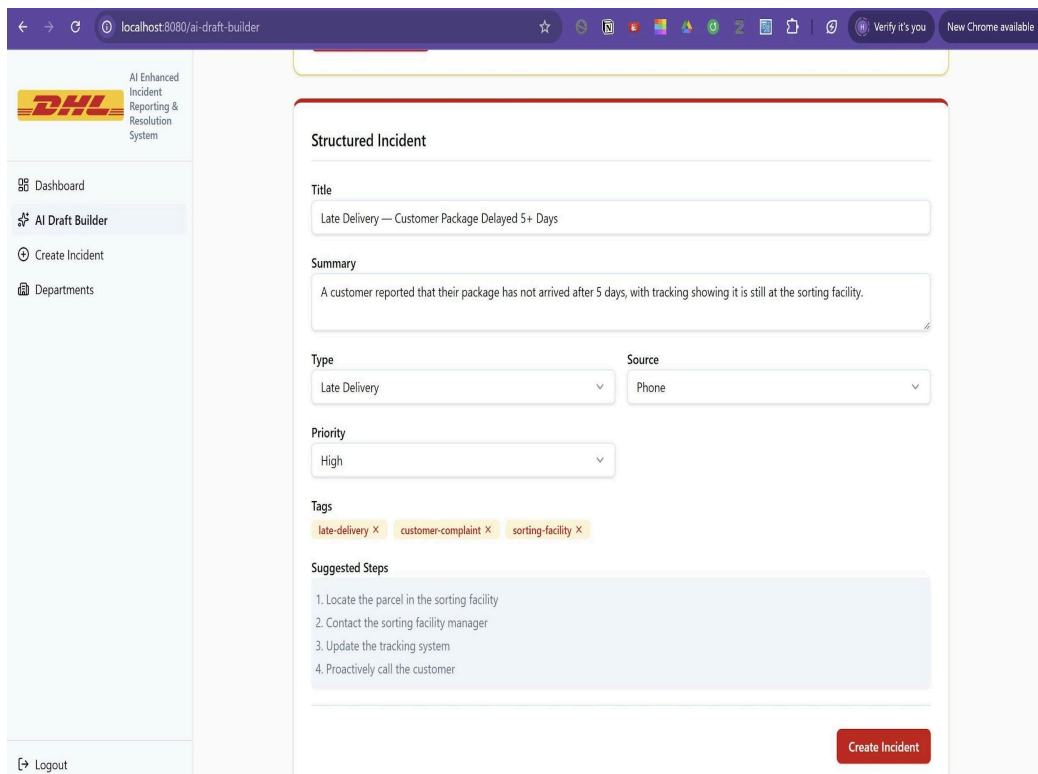
The screenshot shows a web browser at localhost:8080/ai-draft-builder. The page title is 'AI Draft Builder' with the subtitle 'Paste raw text and let AI organize it into a structured incident report.' The form includes a 'Raw Incident Text' field with a placeholder 'Paste email, chat logs, or notes here...'. Below the field is an 'Analyze with AI' button. The left sidebar is identical to the previous screenshot, with 'AI Draft Builder' now active in the navigation menu.

Figure 5: AI Draft Builder — Raw text input interface

5.6 AI Draft Builder — AI Analysis Result

After analysis, Claude AI generates a fully structured incident: Title, Summary, auto-detected Type (Late Delivery), Source (Phone), Priority (High), relevant Tags (late-delivery,

customer-complaint, sorting-facility), and Suggested Resolution Steps. All fields are editable before creating the incident.



The screenshot displays the 'AI Draft Builder' interface for creating a structured incident. The left sidebar contains navigation links: Dashboard, AI Draft Builder (active), Create Incident, and Departments. The main content area is titled 'Structured Incident' and includes the following fields:

- Title:** Late Delivery — Customer Package Delayed 5+ Days
- Summary:** A customer reported that their package has not arrived after 5 days, with tracking showing it is still at the sorting facility.
- Type:** Late Delivery (dropdown)
- Source:** Phone (dropdown)
- Priority:** High (dropdown)
- Tags:** late-delivery x, customer-complaint x, sorting-facility x
- Suggested Steps:**
 1. Locate the parcel in the sorting facility
 2. Contact the sorting facility manager
 3. Update the tracking system
 4. Proactively call the customer

A red 'Create Incident' button is located at the bottom right of the form.

Figure 6: AI Draft Builder — AI-generated structured incident with tags and resolution steps

5.7 Incident Detail Page

The incident detail page shows full information with status badges (draft, medium, Customer Complaint, Email), description, creation timestamp, an AI Conflict Check section for detecting inconsistencies, Update Status control (Draft → Reviewed → Resolved), and a History timeline showing all status changes.

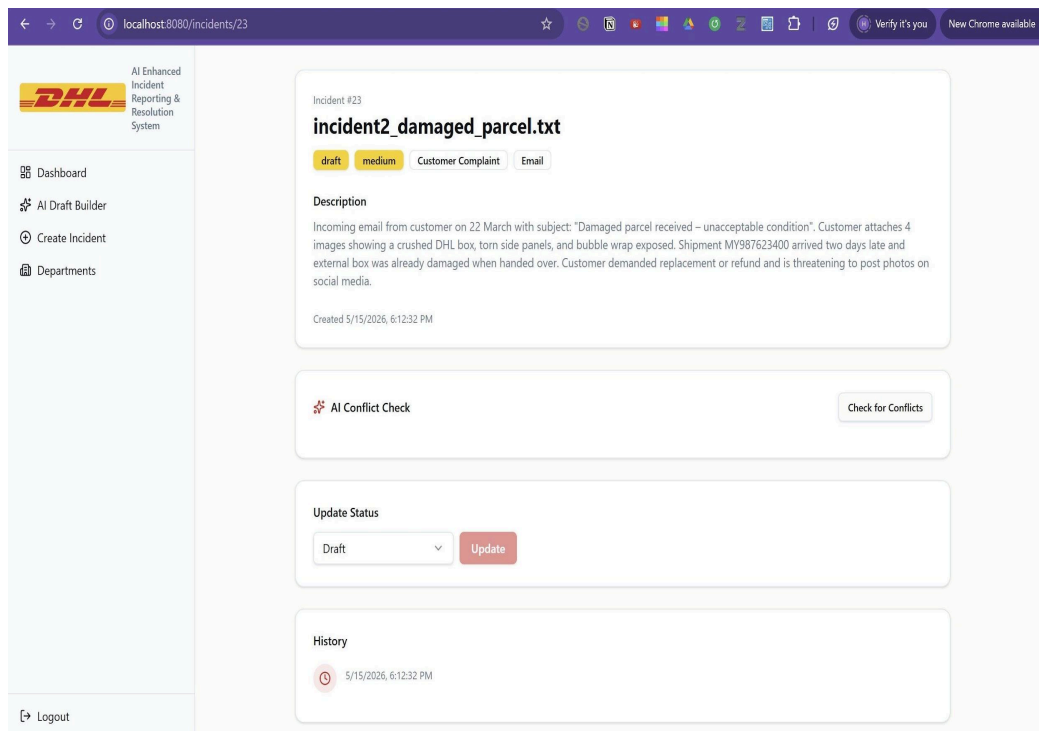


Figure 7: Incident Detail Page — Full view with AI Conflict Check and status history

5.8 Departments Page

The Departments page displays all 5 DHL departments as cards: Customer Support, Warehouse Operations, Finance, IT Support, and Logistics. Departments are used for incident assignment and routing to the appropriate team.

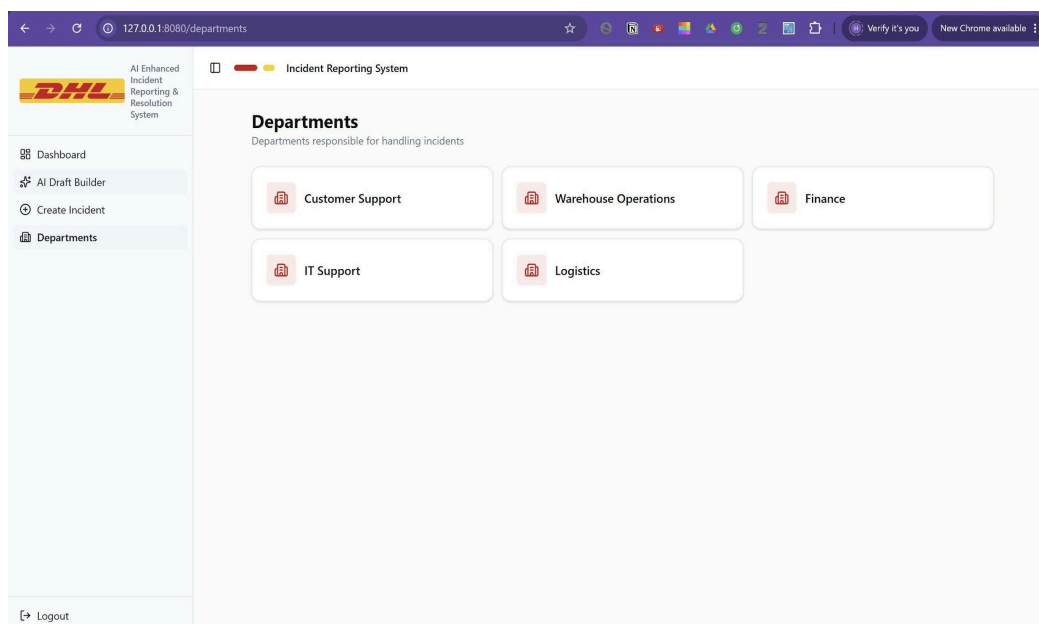


Figure 12: Departments Page — 5 DHL departments for incident assignment

6. User Manual

6.1 Prerequisites

- PHP 8.4+ with extensions: pdo_pgsql, pgsql, mbstring, fileinfo, openssl
- Composer (PHP package manager)
- PostgreSQL 16+
- Node.js 18+ and npm
- UiPath Studio 2026 (Community Edition)
- Git

6.2 Backend Setup

```
git clone https://github.com/Hanan20x/dhl-incident-reporting-system.git
cd backend && composer install
```

Configure .env with PostgreSQL credentials:

```
DB_CONNECTION=pgsql | DB_HOST=127.0.0.1 | DB_PORT=5432 |
DB_DATABASE=dhl_incident_db | DB_USERNAME=postgres
php artisan key:generate && php artisan migrate
php artisan serve
```

6.3 Frontend Setup

```
cd frontend && npm install && npm run dev
```

Open <http://localhost:8080> in your browser.

Login credentials: Email: admin@dhl.com | Password: password123

6.4 Using the System

Step 1 — Login at <http://localhost:8080/login>

Step 2 — View Dashboard to see all incidents with search and filter options

Step 3 — Use AI Draft Builder to paste raw text and let AI structure the incident

Step 4 — Use Create Incident for manual incident reporting with file attachment

Step 5 — Click any incident to view details, run AI Conflict Check, update status

Step 6 — View Departments page to see all 5 DHL departments

6.5 Running the RPA Workflow

Step 1 — Create folder C:\DHL_Incidents and place .txt incident files inside

Step 2 — Open rpa/DHL_Incident_RPA project in UiPath Studio

Step 3 — Ensure Laravel server is running on <http://localhost:8000>

Step 4 — Click Run in UiPath Studio

Step 5 — Check Output panel for processing logs

Step 6 — Check email inbox for summary report with totals: created, skipped, failed